

HealthTracker

Rastreador Personal de Salud

Por: Angel David Quijano, Cesar David Carranza, Miguel Diaz



Descripción General

Proposito: El proyecto consiste en realizar una aplicación que evalúe y registre nuestro estado de salud, haga diagnósticos con respecto a esa información y graficarla para mostrar la tendencia de como estaria nuestra salud en el futuro.

Composición



Registro

Datos del paciente con biométricos
con fecha y hora



Analisis

Tendencias con
cálculos estadísticos y proceso de datos



Visualización

GUI intuitiva
, Graficas y exportación CSV

Arquitectura del Proyecto



Interface

LoginWindow · MainWindow
MetricInputDialog · ChartsWidget



Logica

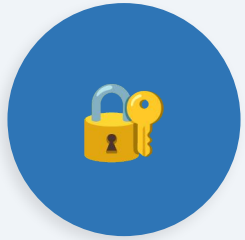
Cuentas · Stats



Database

DatabaseManager · UserDAO
MetricDAO ·

Flujo de la Aplicación



Login

Verificación de
credenciales



Dashboard

3 pestañas:
Registros · Stats · Perfil



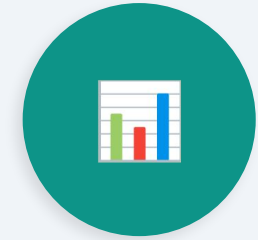
Nueva medición

Diálogo con
filtro de campos



MetricDAO

INSERT en
SQLite3



Estadísticas

Cálculo y
gráfica actualizada

Modelo de Base de Datos (SQLite3)

users

 **id**
username
password (hash)
height_cm
age
sex
goal

weight_logs

 **id**
 **user_id** → users
value_kg
timestamp

blood_pressure_logs

 **id**
 **user_id** → users
systolic
diastolic
timestamp

glucose_logs

 **id**
 **user_id** → users
glucose_mg_dl
context
timestamp

```

DatabaseManager::DatabaseManager()
{
}

bool DatabaseManager::openDatabase(const QString& path)
{
    if (QSqlDatabase::contains("healthtracker_connection")) {
        database = QSqlDatabase::database("healthtracker_connection");
    } else {
        database = QSqlDatabase::addDatabase("QSQLITE", "healthtracker_connection");
    }

    database.setDatabaseName(path);

    if (!database.open()) {
        qDebug() << "Error al abrir la base de datos:" << database.lastError().text();
        return false;
    }

    QSqlQuery query(database);
    query.exec("PRAGMA foreign_keys = ON;");

    qDebug() << "Base de datos creada o abierta correctamente.";
    return true;
}

```

```

bool UserDAO::createUser(const QString& name,
                        const QString& email,
                        const QString& password)
{
    if (emailExists(email)) {
        qDebug() << "El correo ya esta registrado.";
        return false;
    }

    QSqlQuery query(database);
    query.prepare(R"(
        INSERT INTO users (name, email, password_hash)
        VALUES (:name, :email, :password_hash);
    )");

    query.bindValue(":name", name.trimmed().isEmpty() ? email.trimmed() : name.trimmed());
    query.bindValue(":email", email.trimmed());
    query.bindValue(":password_hash", hashPassword(password));

    if (!query.exec()) {
        qDebug() << "Error creando usuario:" << query.lastError().text();
        return false;
    }

    qDebug() << "Usuario creado correctamente.";
    return true;
}

```

```

bool MetricDAO::deleteWeightLog(int logId)
{
    QSqlQuery query(database);

    query.prepare(R"(
        DELETE FROM weight_logs
        WHERE id = :id;
    )");

    query.bindValue(":id", logId);

    if (!query.exec()) {
        qDebug() << "Error eliminando registro de peso:" << query.lastError().text();
        return false;
    }

    qDebug() << "Registro de peso eliminado correctamente.";
    return true;
}

```

Lógica de Cálculo

Estructura de las Cuentas necesarias

TMB (Hombre)	$(10 \times \text{kg}) + (6.25 \times \text{cm}) - (5 \times \text{edad}) + 5$
TMB (Mujer)	$(10 \times \text{kg}) + (6.25 \times \text{cm}) - (5 \times \text{edad}) - 161$
TDEE	TMB $\times$ factor actividad
Bajo	TMB $\times 1.20$
Moderado	TMB $\times 1.55$
Alto	TMB $\times 1.725$
Déficit	TDEE - 500 kcal
Mantenimiento	TDEE
Superávit	TDEE + 300 kcal
IMC	$\text{kg} / (\text{m})^2$

Estrucutra de Stats

<code>average()</code>	La sumatoria de la información / cantidad
<code>minimum()</code>	Número más pequeño
<code>maximum()</code>	Numero más grande
<code>linearTrend()</code>	Regresión lineal
<code>pendiente m</code>	
<code>intercepto b</code>	
<code>isRising</code>	pendiente > 0

```

double Cuentas::calcularBMR(double peso, double altura, int edad, Sexo sexo)
{
    if (sexo == Sexo::Masculino) {
        return 10.0 * peso + 6.25 * altura - 5.0 * edad + 5.0;
    }
    return 10.0 * peso + 6.25 * altura - 5.0 * edad - 161.0;
}

double Cuentas::calcularTDE(double bmr, nivel_actividad actividad)
{
    switch (actividad) {
        case nivel_actividad::Bajo:
            return bmr * 1.2;
        case nivel_actividad::Moderado:
            return bmr * 1.55;
        case nivel_actividad::Alto:
            return bmr * 1.9;
    }
    return bmr;
}

double Cuentas::calcularMeta(double tde, objetivo obj)
{
    switch (obj) {
        case objetivo::Perder_peso:
            return tde - 500.0;
        case objetivo::Mantener_peso:
            return tde;
        case objetivo::Ganar_peso:
            return tde + 500.0;
    }
    return tde;
}

```

```

double Cuentas::calcularBMI(double peso, double altura)
{
    if (altura <= 0.0) {
        return 0.0;
    }
    return peso / ((altura / 100.0) * (altura / 100.0));
}

QString Cuentas::interpretarBMI(double bmi)
{
    if (bmi <= 0.0) {
        return "Sin datos";
    }
    if (bmi < 18.5) {
        return "Bajo peso";
    }
    if (bmi < 25.0) {
        return "Normal";
    }
    if (bmi < 30.0) {
        return "Sobrepeso";
    }
    return "Obesidad";
}

```

```

double Stats::ave(const QVector<double> &data)
{
    if (data.isEmpty()) {
        return 0.0;
    }
    const double sum = std::accumulate(data.begin(), data.end(), 0.0);
    return sum / data.size();
}

double Stats::min(const QVector<double> &data)
{
    if (data.isEmpty()) {
        return 0.0;
    }
    return *std::min_element(data.begin(), data.end());
}

double Stats::max(const QVector<double> &data)
{
    if (data.isEmpty()) {
        return 0.0;
    }
    return *std::max_element(data.begin(), data.end());
}

```


Interface

Ventanas

- Ventana de login
- Ventana principal
 - Ingresos de datos
 - Grafica de datos
 - Información del usuario

Contenido

- Login y registro
- Ingresar datos y importación de los datos
- Gráfica de los datos
- Información del usuario

HealthTracker — Login

HealthTracker

Name (for registration):

Your name

Email:

example@email.com

Password:

Your password

Log In

Register

HealthTracker — m

Nueva medición

Registros

Estadísticas

Perfil

Mostrar:

Peso

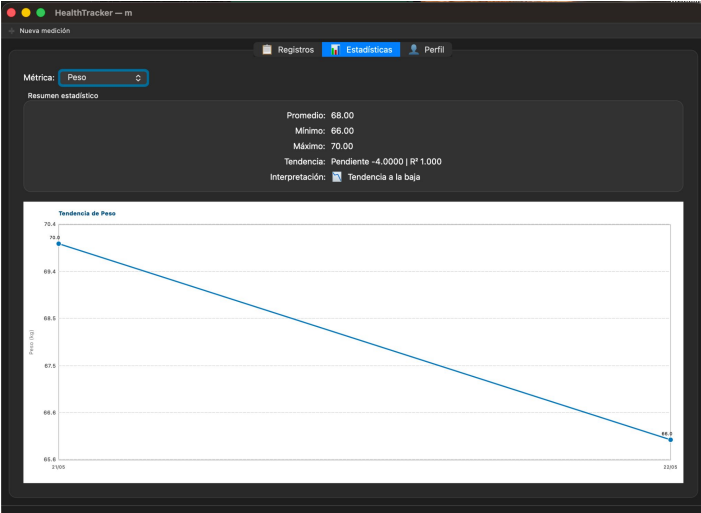
ID | Peso (kg) | Fecha

Nueva medición

Actualizar

Eliminar seleccionado

Exportar CSV



Registros

Estadísticas

Perfil

Datos del perfil

Edad:

30

Estatura: 170.0 cm

Sexo: Masculino

Actividad: Bajo

Objetivo: Mantener peso

Guardar perfil

Calcular BMR / calorías

Resultados

BMR (kcal/día): 1577.50 kcal/día

TDEE (kcal/día): 1893.00 kcal/día

Objetivo calórico: 1893.00 kcal/día

IMC: 22.84

Interpretación IMC: Normal

Referencia

Frankenfield, David, et al. "Comparison of Predictive Equations for Resting Metabolic Rate in Healthy Nonobese and Obese Adults: A Systematic Review." *Journal of the American Dietetic Association*, vol. 105, no. 5, May 2005, pp. 775–789. doi:10.1016/j.jada.2005.02.005.

"Mifflin-St Jeor Equation." *Medscape Reference*, QxMD, reference.medscape.com/calculator/846/mifflin-st-jeor-equation. Accessed 21 May 2026.

Webb, Scott. "Simple Linear Regression." *Mostly Harmless Statistics*, Statistics LibreTexts, 12 Mar. 2023, stats.libretexts.org/Bookshelves/Introductory_Statistics/Mostly_Harmless_Statistics_(Webb)/12:_Correlation_and_Regression/12.02:_Simple_Linear_Regression.

"12.3 Simple Linear Regression." *STAT 200: Elementary Statistics*, Pennsylvania State University, online.stat.psu.edu/stat200/lesson/12/12.3. Accessed 21 May 2026.

Devore, Jay L. *Probability and Statistics for Engineering and the Sciences*. 9th ed., Cengage Learning, 2016, pp. 469–478.

